

UNIVERSIDAD EAFIT
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA
ARQUITECTURA DE SOFTWARE 2026

Tutorial 05: El Escudo Arquitectónico

Nginx, Gunicorn y las bases del Strangler Pattern en AWS

Nicolás Rodríguez

Profesor: Nicolás Ramírez Vélez

30 de septiembre de 2026

1. Evidencias de entrega (AWS EC2)

1.1. Evidencia de Enrutamiento (Browser)

`http://<IP-EC2>/api/v1/comprar/`

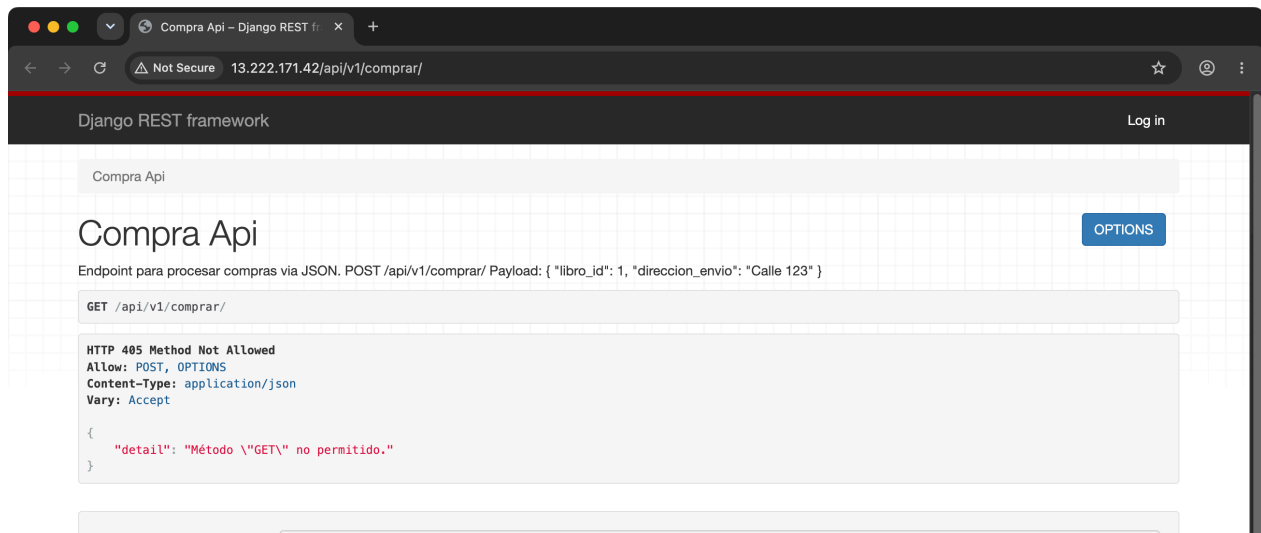


Figura 1: `http://13.222.171.42/api/v1/comprar/` — sin puerto 8000: Nginx enruta el tráfico del puerto 80 hacia Django/Gunicorn.

1.2. Evidencia de Aislamiento (Terminal local)

`http://<IP-EC2>:8000`

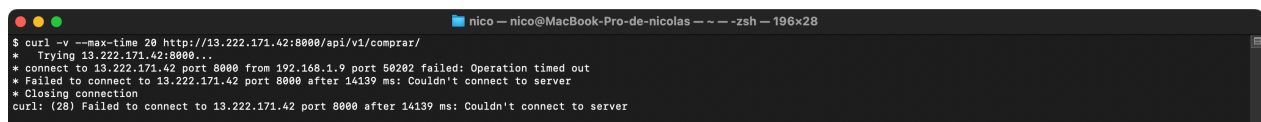


Figura 2: Terminal local: curl a `13.222.171.42:8000` termina en timeout (curl 28). Django está aislado; solo se llega por el proxy inverso.

1.3. Evidencia de Orquestación (Terminal AWS)

docker ps

```

[ec2-user@ip-172-31-21-153 tienda-arquitectura-2026]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
1a66441d1df3   nginx:1.25-alpine                  "/docker-entrypoint..." About a minute ago Up About a minute   0.0.0.0:80->80/tcp, :::80->80/tcp   tienda-arquitectura-2026-nginx-1
630d97f085ec   tienda-arquitectura-2026-web       "sh -c 'python manag..." About a minute ago Up About a minute   5432/tcp                             tienda-arquitectura-2026-web-1
757726804e5    postgres:15                        "docker-entrypoint.s..." About a minute ago Up About a minute (healthy)         tienda-arquitectura-2026-db-1
[ec2-user@ip-172-31-21-153 tienda-arquitectura-2026]$

```

Figura 3: Terminal de la instancia EC2 (ec2-user@ip-172-31-21-153): nginx, web y db en estado Up.

2. Paso 1: Gunicorn

requirements.txt

```

asgiref==3.12.1
Django==5.2.17
django-filter==26.1
djangorestframework==3.18.1
gunicorn==26.2.0
Markdown==3.11
psycopg2-binary==2.9.13
sqlparse==0.6.0
whitenoise==6.12.0

```

Además de gunicorn agregué **WhiteNoise**: Gunicorn no sirve archivos estáticos, y sin él la interfaz de DRF y el admin cargan sin CSS. El contenedor corre collectstatic al arrancar y WhiteNoise los sirve desde staticfiles/.

3. Paso 2: nginx/nginx.conf

nginx/nginx.conf

```

# Registro que muestra a QUE backend se envia cada peticion (evidencia del Strangler)
log_format strangler '$remote_addr [$time_local] "$request" $status '
                        '-> upstream=$upstream_addr ($upstream_response_time s)';

upstream django_v1 {
    server web_django:8000;
}

upstream flask_v2 {
    server pagos_flask:5000;
}

server {
    listen 80;
    access_log /var/log/nginx/access.log strangler;

    location = /favicon.ico { access_log off; log_not_found off; }

    # RUTA VIEJA: Sigue yendo a Django
    location /api/v1/ {
        proxy_pass http://django_v1;
        proxy_set_header Host $host;
    }
}

```

```

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

# RUTA NUEVA: Estrangulamos la funcion de compra y la mandamos a Flask
location /api/v2/comprar {
    proxy_pass http://flask_v2;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

# Todo lo que aun no se ha migrado (HTML, admin, estaticos) sigue en el monolito
location / {
    proxy_pass http://django_v1;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
}

```

4. Paso 3: docker-compose.yml

docker-compose.yml

```

services:
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=tienda_db
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=secret_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d tienda_db"]
      interval: 3s
      timeout: 3s
      retries: 20

  web_django: # El Monolito (v1)
    build: .
    command: sh -c "python manage.py migrate && python manage.py seed_tienda && python manage.py collectstatic --noinput && gunicorn Tienda.wsgi:application --bind 0.0.0.0:8000"
    volumes:
      - ./app
    depends_on:
      db:
        condition: service_healthy
    environment:
      - DB_NAME=tienda_db
      - DB_USER=postgres
      - DB_PASSWORD=secret_password
      - DB_HOST=db
      - DB_PORT=5432

  pagos_flask: # El Microservicio (v2) - EL ESTRANGULADOR
    build: ./microservicio_pagos
    ports:
      - "5000:5000" # Solo para pruebas iniciales, luego lo cerraremos

```

```
nginx:
  image: nginx:1.25
  ports:
    - "80:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
  depends_on:
    - web_django
    - pagos_flask

volumes:
  postgres_data:
```

El proyecto se llama `Tienda` (no `tienda_project`), así que el comando es `gunicorn Tienda.wsgi:application`. El servicio `web` ya no tiene `ports`: vive solo en la red interna de Docker y únicamente Nginx puede hablarle.

5. Paso 4: Despliegue en AWS EC2

Terminal EC2

```
cd tienda-arquitectura-2026
# (codigo actualizado subido por scp; con GitHub seria: git pull origin main)
docker compose up -d --build
docker ps
```

1. EC2 → Security Groups → grupo de la instancia → **Edit inbound rules**.
2. **Add rule**: Type **HTTP** · Port **80** · Source **0.0.0.0/0** → Save rules.
3. Para que la evidencia 1.2 muestre **timeout** (y no "connection refused"), **eliminar la regla del puerto 8000** que se creó en el Tutorial 04: así el firewall de AWS descarta los paquetes en silencio.

6. Por qué esta arquitectura

`runserver` es para desarrollo: un solo hilo, sin endurecimiento y avisando en consola que no se use en producción. Gunicorn maneja múltiples workers y Nginx absorbe el tráfico, oculta la topología interna y se convierte en la **fachada** del Strangler Pattern: el día que un endpoint migre a otro servicio, se cambia una `location` en Nginx y el cliente no se entera.

Anexo: validación previa en local (misma arquitectura)

Antes de subirlo a la nube probé exactamente el mismo `docker-compose.yml` en una máquina virtual Linux local (Colima) con IP propia 192.168.64.2, que hace el papel de la IP pública de EC2. Los tres criterios se cumplen:

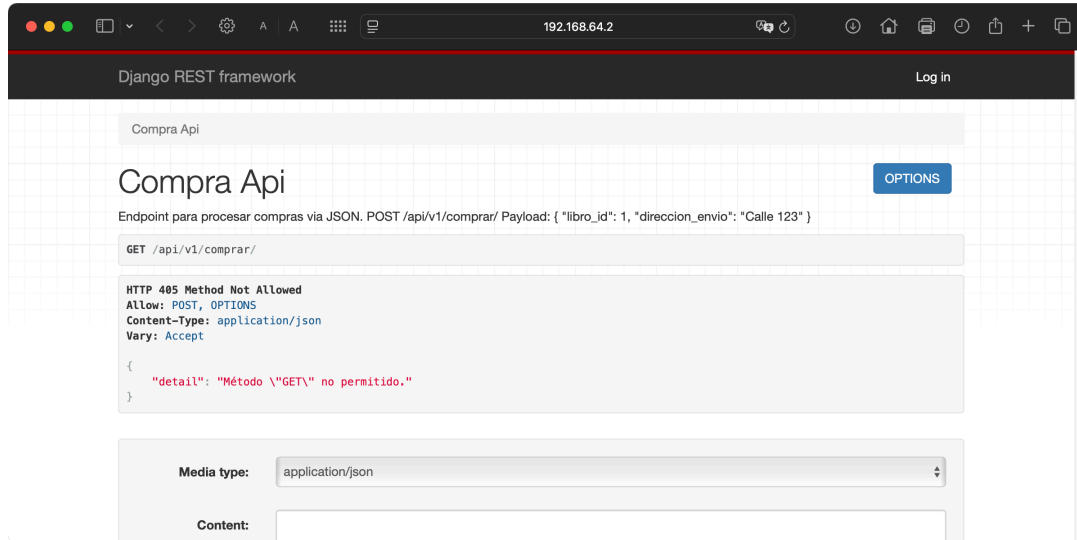


Figura 4: Enrutamiento: `http://192.168.64.2/api/v1/comprar/` responde la API sin puerto 8000 — Nginx enruta por el puerto 80.

```
$ curl -v --max-time 15 http://192.168.64.2:8000/api/v1/comprar/
* Trying 192.168.64.2:8000...
* connect to 192.168.64.2 port 8000 from 192.168.64.1 port 49449 failed: Connection refused
* Failed to connect to 192.168.64.2 port 8000 after 1010 ms: Couldn't connect to server
* Closing connection
curl: (7) Failed to connect to 192.168.64.2 port 8000 after 1010 ms: Couldn't connect to server

$ curl -s -o /dev/null -w "HTTP %{http_code} servido por %{header{server}}\n" http://192.168.64.2/api/v1/comprar/
HTTP 405 servido por nginx/1.25.5
```

Figura 5: Aislamiento: el puerto 8000 rechaza la conexión, mientras el 80 responde con cabecera `Server: nginx/1.25.5`.

```
$ docker ps
CONTAINER ID   IMAGE                STATUS              PORTS                               NAMES
3f22f0a56e78   nginx:1.25-alpine    Up 40 seconds       0.0.0.0:80->80/tcp, [::]:80->80/tcp  proyectotienda-tutoriales-nginx-1
1df57052f2f9   proyectotienda-tutoriales-web    Up 40 seconds                               proyectotienda-tutoriales-web-1
0587df21d686   postgres:15          Up 44 seconds (healthy)  5432/tcp                               proyectotienda-tutoriales-db-1
```

Figura 6: Orquestación: nginx, web y db en estado Up. Solo nginx publica un puerto (80); web no expone nada.

```
curl -i -u nicolas:**** -d '{"libro_id": 3, ...}' http://192.168.64.2/api/v1/comprar/
HTTP/1.1 201 Created
Server: nginx/1.25.5
{"estado":"exito","mensaje":"Orden creada. Total: 196.35"}
```

La compra también funciona de punta a punta a través del proxy: 201 Created, y la cabecera confirma que quien da la cara es Nginx.